A Cloud-Based Data Leakage Prevention System Using Rule-Based Content Inspection

TEAM MEMBERS :

SABARIRAJA. S (620822205090)

RANJITH. P (620822205084)

SHANTHOSH. M (620822205098)

ROHITH. V (620822205088)

GUIDE : JAYANTHI. R,AP/IT

Abstract

- Cloud computing has become a primary platform for data storage and file sharing, increasing the risk of sensitive data leakage. Data leakage may occur due to user negligence, misconfiguration, or malicious intent, often before users realize the exposure. This project proposes a cloud-integrated Data Leakage Prevention (DLP) system that inspects data content before it is uploaded to the cloud.

- The system uses rule-based content inspection to identify sensitive information such as email addresses, passwords, and predefined confidential patterns. If sensitive content is detected, the upload is blocked and the event is logged. Cryptographic hashing is integrated to ensure file integrity and detect unauthorized modifications.

- All inspection and decision-making processes occur within the user environment, preserving data privacy and preventing raw data exposure to the cloud provider.

- The proposed system is lightweight, deterministic, and suitable for real-time cloud usage, offering an effective and privacy-aware solution for preventing accidental and intentional data leakage.

Introduction

- Cloud storage is widely used for personal and organizational data
- Sensitive data is often uploaded without awareness of security risks
- Traditional cloud security mechanisms focus on server-side controls
- Server-side inspection compromises privacy and requires trust in cloud providers
- Data Leakage Prevention (DLP) aims to stop unauthorized data exposure
- There is a need for user-side, privacy-preserving DLP solutions

PROBLEM STATEMENT:

- Users unknowingly upload sensitive information to cloud platforms
- Cloud providers cannot always detect confidential data correctly
- Existing DLP solutions rely on centralized inspection
- Centralized systems introduce privacy risks and single points of failure
- No simple mechanism exists to verify file integrity after upload
- A secure, lightweight, user-side DLP system is required

OBJECTIVES:

- To design a user-side Data Leakage Prevention system
- To detect sensitive data before cloud upload
- To prevent accidental and malicious data leakage
- To preserve user privacy during content inspection
- To verify file integrity using cryptographic hashing
- To maintain low computational overhead for real-time use

EXISTING SYSTEM:

- Data is uploaded directly to the cloud
- Security controls are applied mostly on the server side
- Cloud providers perform centralized content inspection
- Users have limited visibility and control
- Integrity checks are often absent or optional

DISADVANTAGES:

- Privacy loss due to server-side data inspection
- Dependency on cloud service providers
- Single point of failure in centralized systems
- Delayed detection of sensitive data leakage
- No reliable mechanism to detect file tampering

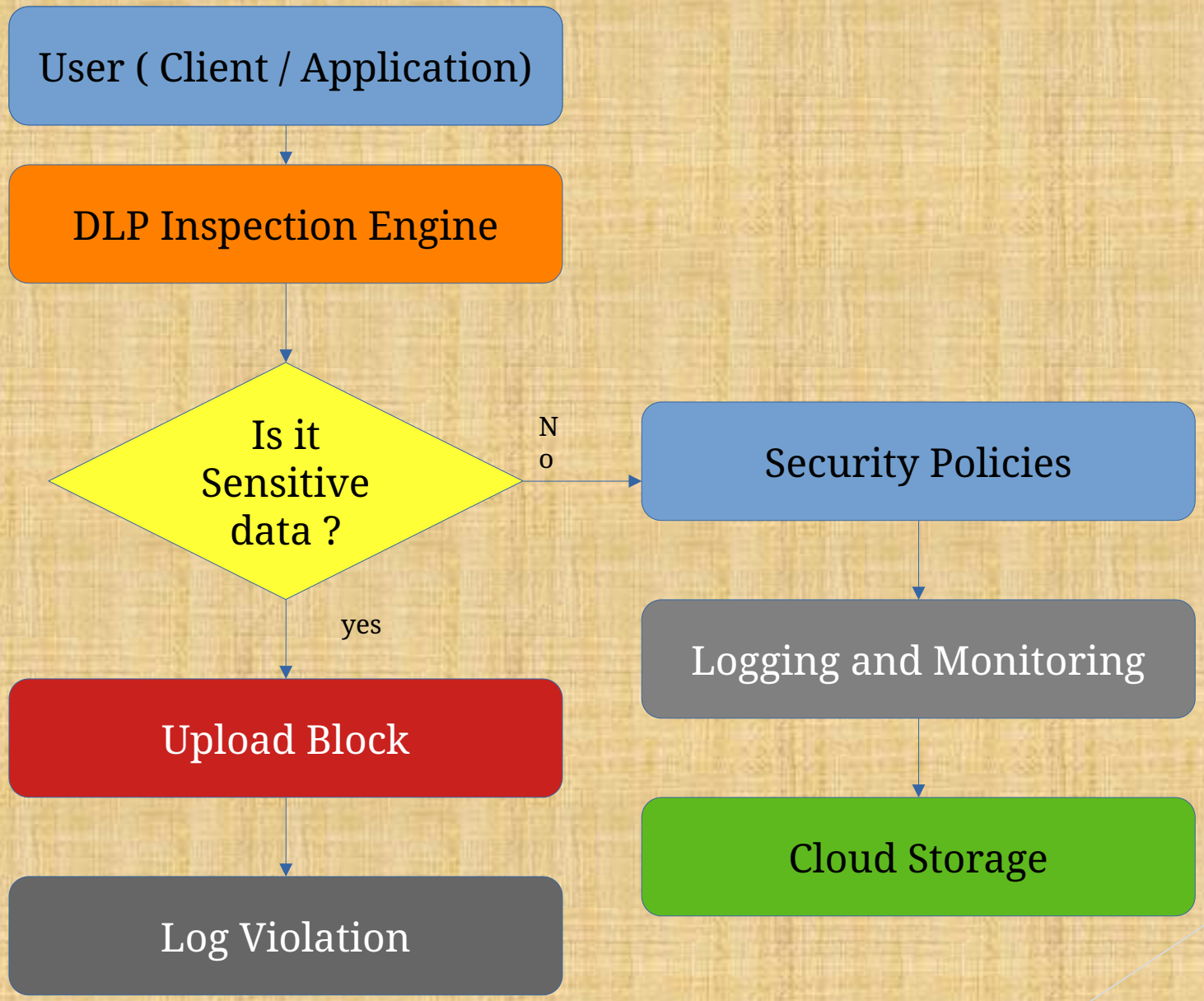
PROPOSED SYSTEM :

- Data inspection performed on the user device
- Rule-based content scanning for sensitive patterns
- Upload blocked if sensitive data is detected
- Cryptographic hash generated for each file
- Hash used to verify file integrity and detect modifications

ADVANTAGES:

- Prevents sensitive data leakage at the source
- Deterministic and transparent behavior
- Lightweight and suitable for real-time operation
- No dependence on third-party inspection
- Improves trust and security in cloud usage

SYSTEM ARCHITECTURE /FLOW DIAGRAM :



MODULES :

- Module 1: Rule-Based Content Inspection
- Module 2: Cryptographic Hashing for Integrity
- Module 3: Secure Violation Logging
- Module 4: User-Defined Security Policies
- Module 5: Performance Optimization & Cloud Integration

Module 1: Rule-Based Content Inspection

- Goal: Automatically detect sensitive data (PII, financial, credentials) in files and payloads
- Approach: Predefined rules using regex patterns, keywords, and context (e.g., “DOB: 01/01/2000”)
- Pipeline: Input → normalization → rule evaluation → list of violations → actions (block/mask/allow)
- Benefits: Simple, explainable, and auditable detection; foundation for all later modules

Module 2: Cryptographic Hashing for Integrity

- Goal: Make DLP records and content fingerprints tamper-evident
- Technique: Secure hash functions (e.g., SHA-256) applied to violation fields and/or content
- Design: Per-record hashes +
  optional hash chaining across log entries
- Outcome:
  Any modification of critical fields is detectable during audits or inve stigations

Module 3: Secure Violation Logging

- Goal: Capture DLP violations in a secure, compliant, and useful way
- Log contents: Timestamp, rule ID, user/service, resource info, action, severity, violation_hash
- Security: Integrity via hashes; confidentiality via masking/truncation and access control on logs
- Value: Reliable evidence for audits, incident response, and tuning DLP rules/policies

CONCLUSION

- Data leakage is a critical challenge in cloud environments
- The proposed system prevents leakage at the user level
- Client-side inspection ensures privacy and control
- Hash-based integrity verification strengthens security
- The solution is practical, lightweight, and extensible
- Future enhancements can add advanced policies and monitoring

THANK YOU !